



LAB 1

MICROPROCESSORS – MICROCONTROLLERS

LECTURER: NGUYỄN HÒA HƯNG
PHAN VĂN SỸ

GROUP: CC03

STUDENT

NAME: HUỖNH TRÍ ĐỨC

ID: 2452269

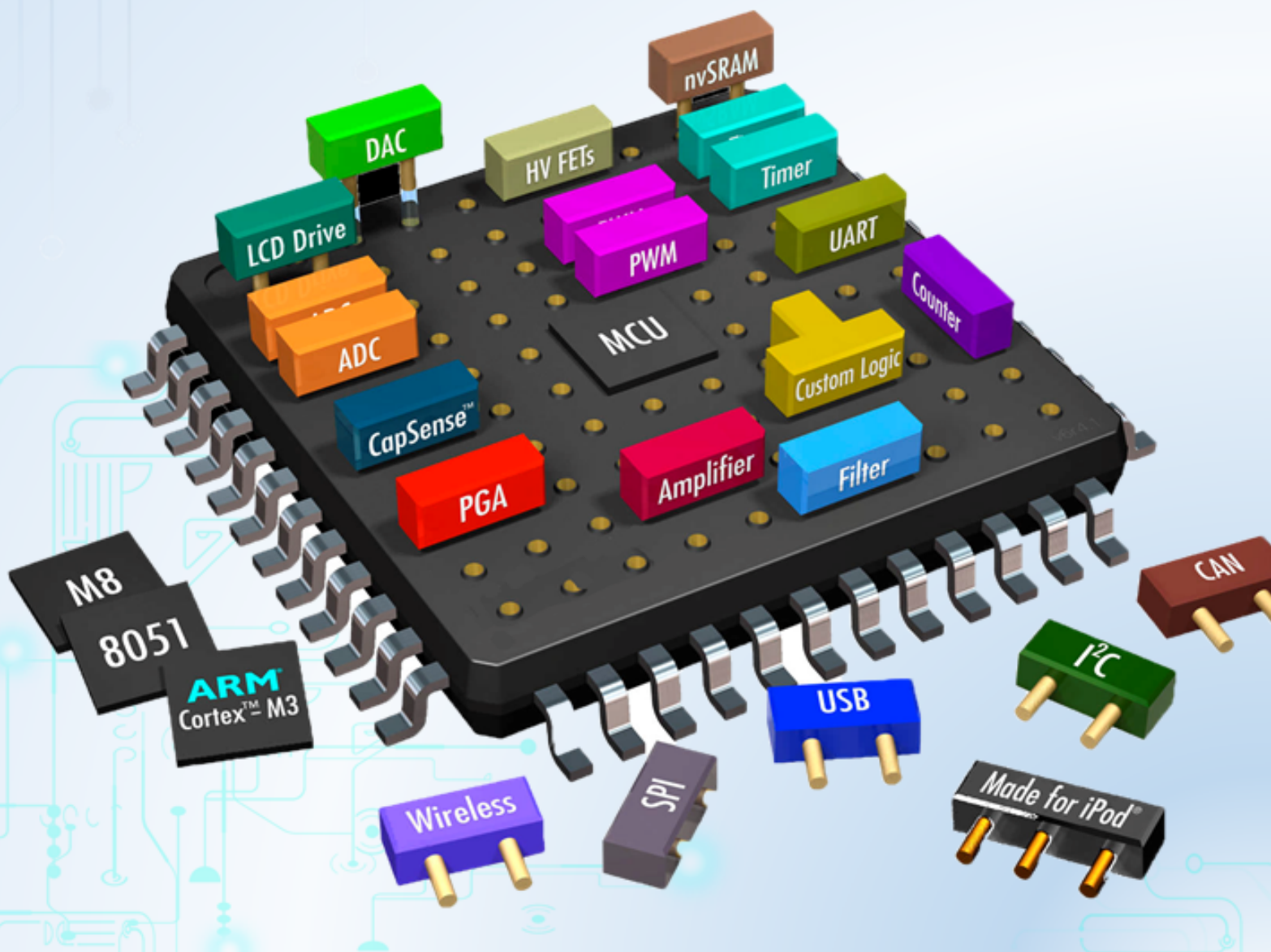
September 18, 2026

Contents

Chapter 1. LED Animations	3
1 Introduction	4
2 First Project on STM32Cube	4
3 Simulation on Proteus	9
4 Exercise and Report	15
4.1 Exercise 1	15
4.2 Exercise 2	18
4.3 Exercise 3	20
4.4 Exercise 4	23
4.5 Exercise 5	25
4.6 Exercise 6	29
4.7 Exercise 7	31
4.8 Exercise 8	32
4.9 Exercise 9	33
4.10 Exercise 10	33

CHAPTER 1

LED Animations



1 Introduction

In this manual, STM32CubeIDE is used as the editor and development environment for programming an ARM microcontroller. STM32CubeIDE is an advanced C/C++ development platform that provides peripheral configuration, code generation, compilation, and debugging features for STM32 microcontrollers and microprocessors.



Figure 1.1: STM32CubeIDE for STM32 programming

After selecting an empty STM32 MCU or MPU, or a preconfigured development board, STM32CubeIDE can automatically generate the initialization code. During development, users can return to the peripheral or middleware configuration and regenerate this code without affecting code placed in the designated user-code regions. This feature simplifies initialization and accelerates application development.

The software used in this laboratory can be downloaded from the following locations:

- STM32CubeIDE: https://ubc.sgp1.digitaloceanspaces.com/BKU_Softwares/STM32/stm32cubeide_1.7.0.zip
- Proteus: https://ubc.sgp1.digitaloceanspaces.com/BKU_Softwares/STM32/Proteus_8.10_SP0_Pro.exe

For remote laboratory sessions, the programs are first simulated in Proteus. The remainder of this manual explains how to:

- create a project in STM32CubeIDE;
- create a project in Proteus;
- simulate the project in Proteus; and
- complete the ten exercises in the report.

2 First Project on STM32Cube

Step 1: Launch STM32CubeIDE. From the File menu, select New, then choose STM32 Project.

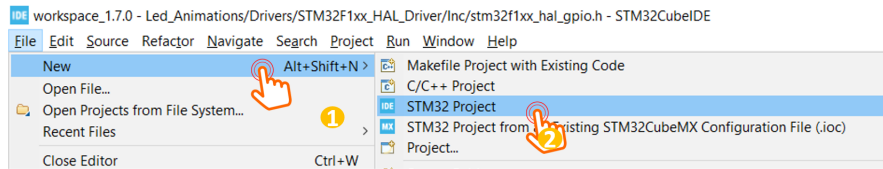


Figure 1.2: Creating a new project in STM32CubeIDE

The IDE may need to download additional packages the first time a project is created.

Step 2: Select STM32F103C6 in the target-selection dialog, then click Next.

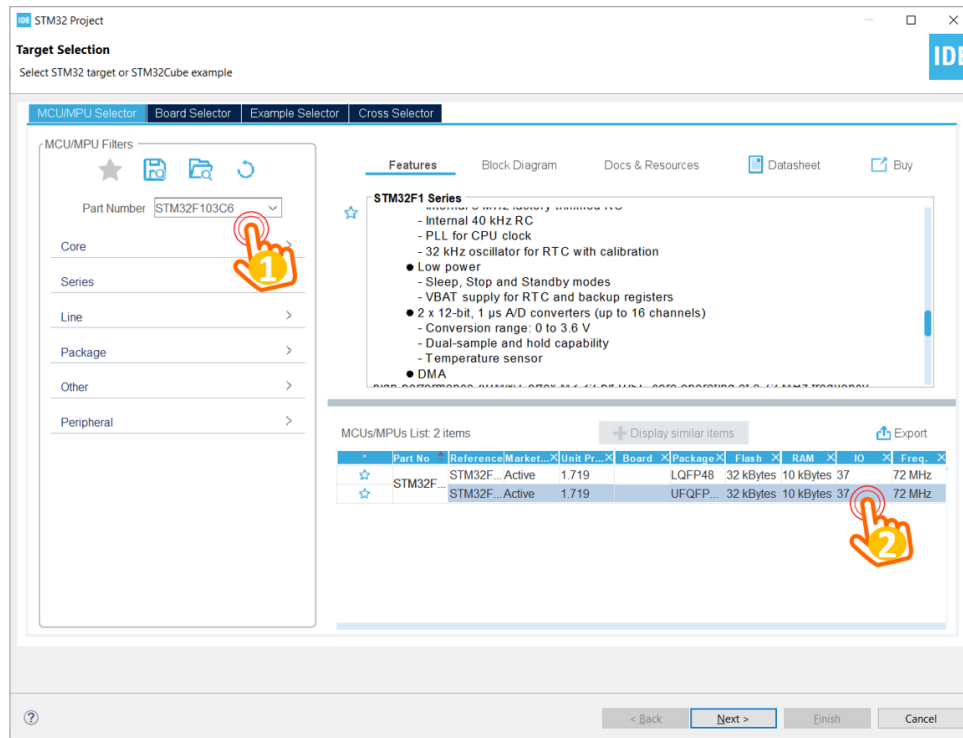


Figure 1.3: Selecting the target device

Step 3: Provide the project name and location.

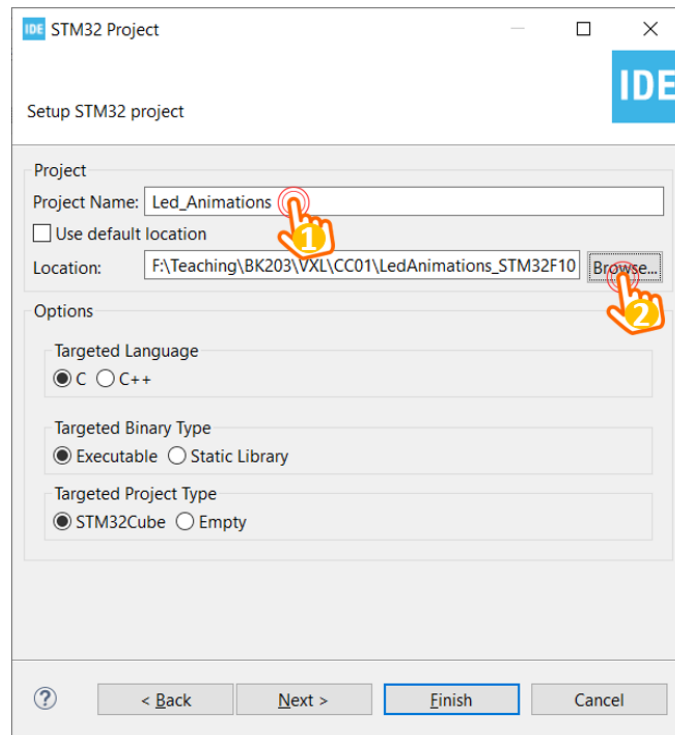


Figure 1.4: Providing the project name and location

The targeted project type must be STM32Cube. If this option is disabled, repeat Step 1. The project path should not contain special characters such as spaces. Click Next when finished.

Step 4: Keep the default firmware version in the final dialog and click Finish.

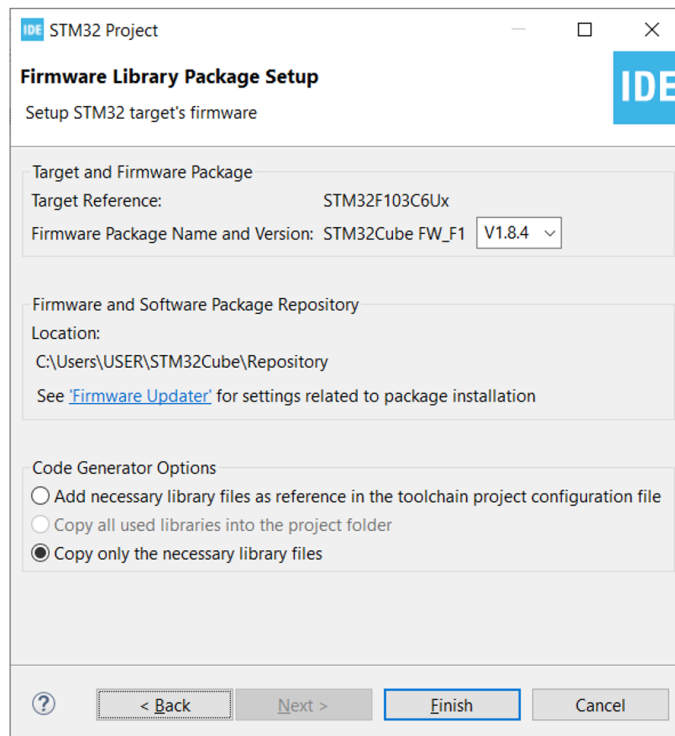


Figure 1.5: Keeping the default firmware version

Step 5: After the project is created, STM32CubeIDE displays the device-configuration view. Select PA5 and set it to GPIO_Output, because this pin will be connected to an LED.

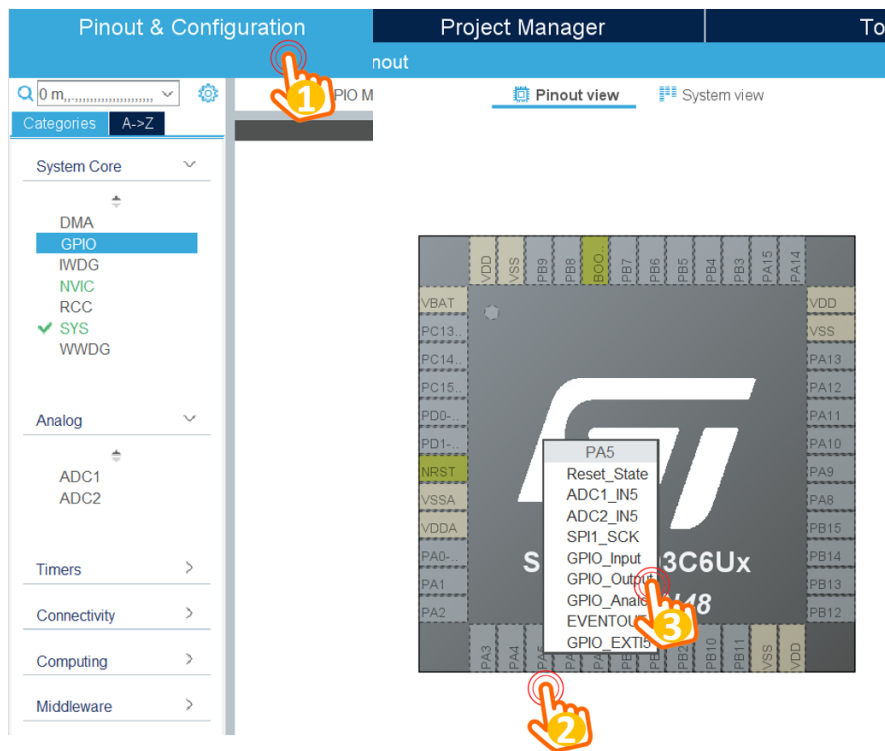


Figure 1.6: Setting PA5 to GPIO output mode

Step 6: Right-click PA5, select Enter User Label, and name the pin, for example, LED_RED. A meaningful label makes the program easier to understand.

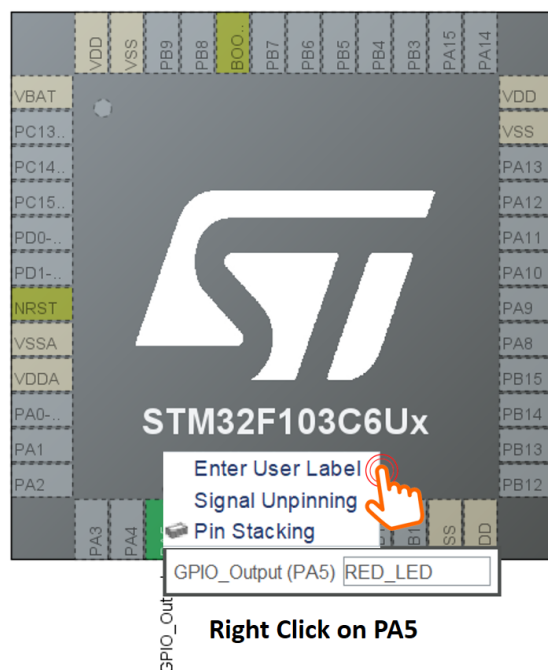


Figure 1.7: Providing a user label for PA5

Save the configuration by pressing Ctrl+S, then click OK to generate the initialization code.

Step 7: Add the following statements to the infinite loop in the main function:

```
1 while (1)
2 {
3     HAL_GPIO_TogglePin(LED_RED_GPIO_Port , LED_RED_Pin);
4     HAL_Delay(1000);
5 }
```

Program 1.1: First blinking-LED program

Place custom code only in the designated USER CODE regions; otherwise, it may be deleted when STM32CubeIDE regenerates the project. The editor suggestions can be opened using Ctrl+Space.

Step 8: Enable HEX-file generation. From the Project menu, select Properties. Navigate to C/C++ Build, Settings, and MCU Post build outputs, then select Convert to Intel Hex file.

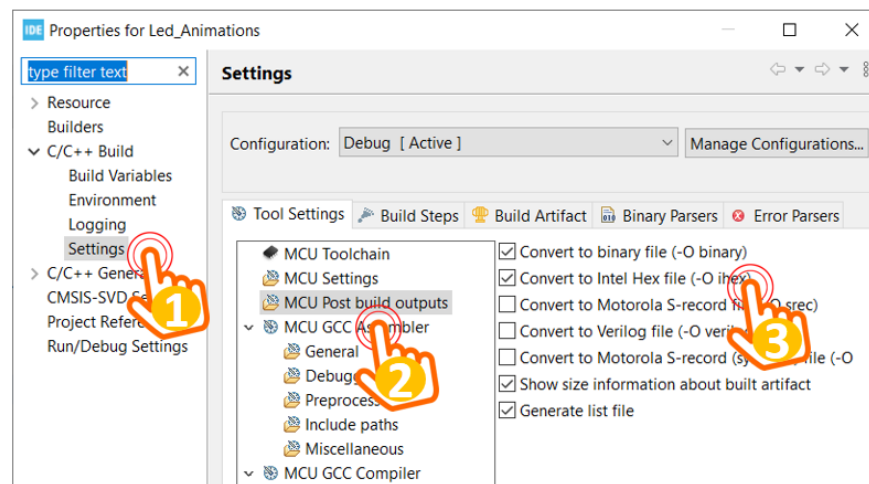


Figure 1.8: Configuring HEX-file output

Step 9: From the Project menu, select Build Project. Check the output console to confirm that the HEX file was generated successfully.

```
22:36:06 **** Incremental Build of configuration Debug for project Led_Animations ****
make -j8 all
arm-none-eabi-size  Led_Animations.elf
   text    data    bss    dec    hex filename
   4596     20    1572    6188    182c Led_Animations.elf
Finished building: default.size.stdout

22:36:06 Build Finished. 0 errors, 0 warnings. (took 272ms)
```

Figure 1.9: Building the project and generating the HEX file

The HEX file is located in the project's Debug directory and will be used for the Proteus simulation. If several projects are open in one workspace, activate the intended project before building and verify the generated output files.

3 Simulation on Proteus

Step 1: Launch Proteus with administrator privileges. From the File menu, select New Project.



Figure 1.10: Creating a new project in Proteus

Step 2: Provide the project name and location, then click Next.

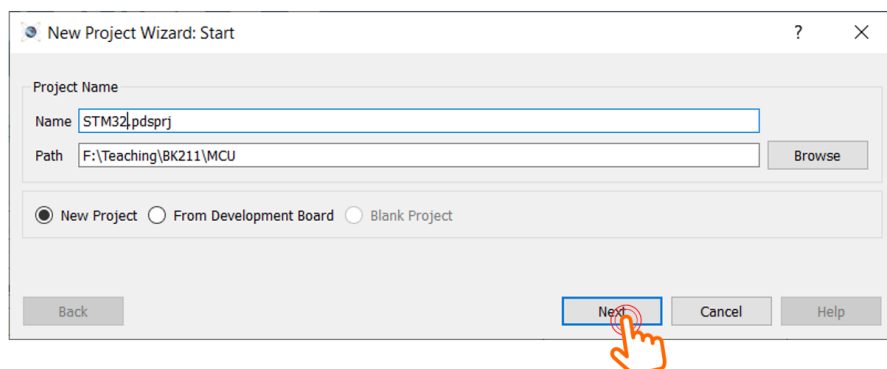


Figure 1.11: Providing the Proteus project name and location

Step 3: Keep the default options in the following dialogs and click Next, because only a schematic is required for this lab.

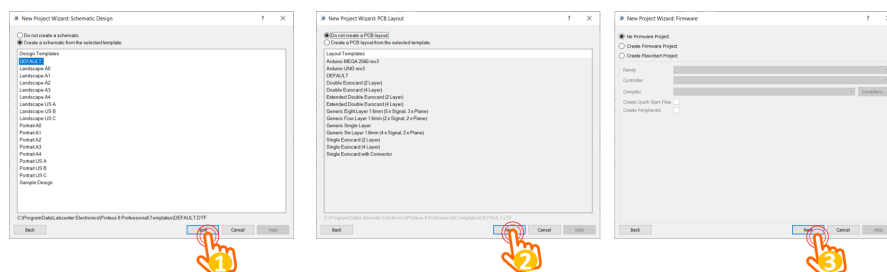


Figure 1.12: Keeping the default project options

Step 4: Click Finish to close the project wizard.

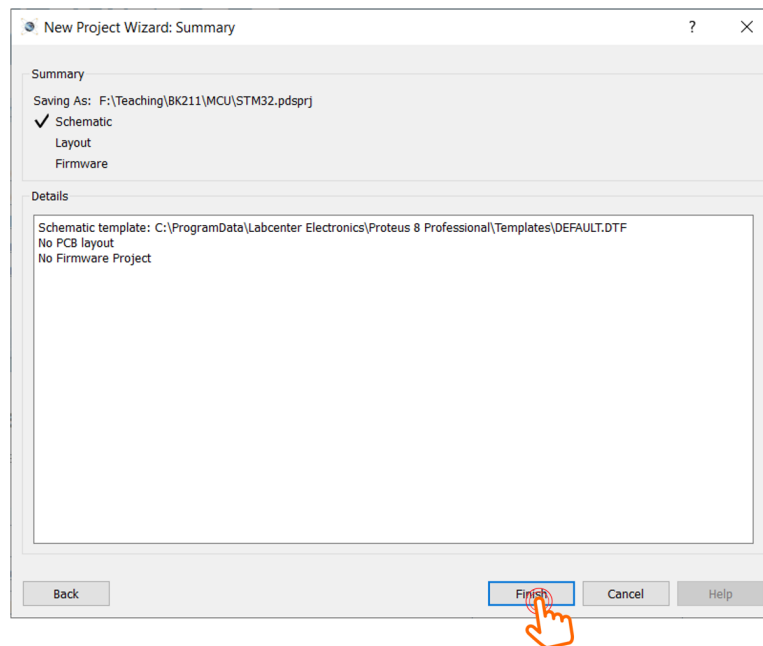


Figure 1.13: Finishing the Proteus project wizard

Step 5: On the schematic page, right-click and select Place, Components, then From Libraries.

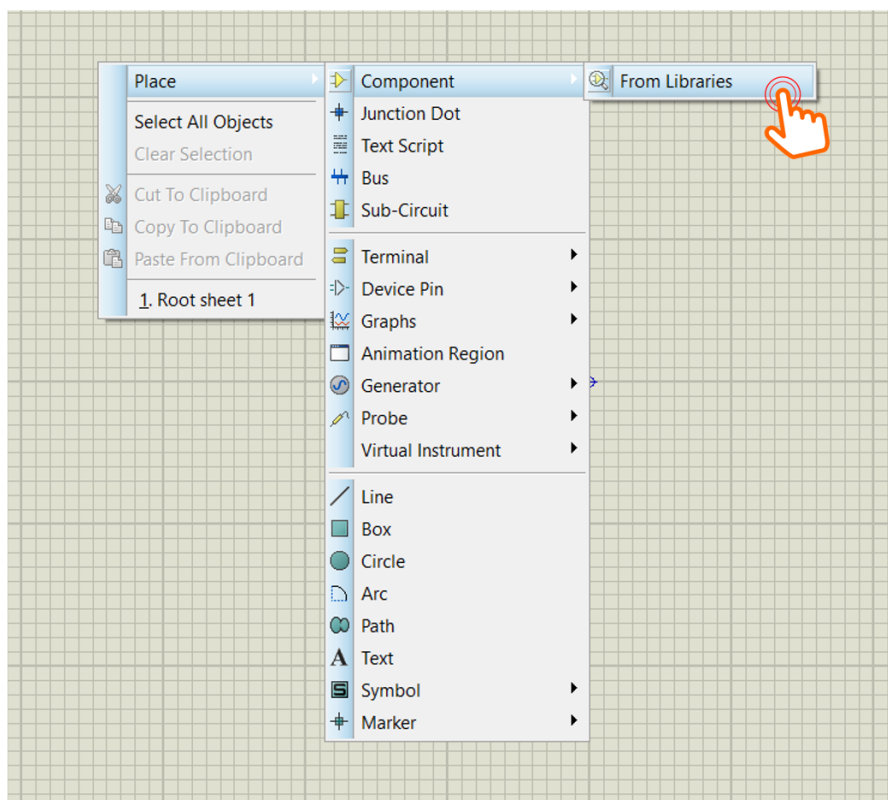


Figure 1.14: Selecting a component from the Proteus library

If Proteus reports that no library is available, restart it using the Run as administrator option.

Step 6: Select STM32F103C6 from the component library. Repeat Step 5 to add an LED-RED. Both components then appear in the DEVICES panel.

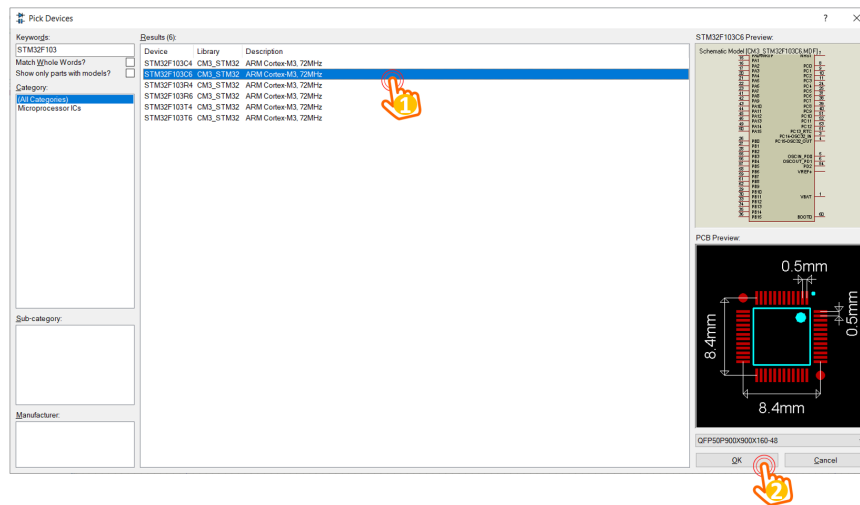


Figure 1.15: Selecting STM32F103C6 from the component library

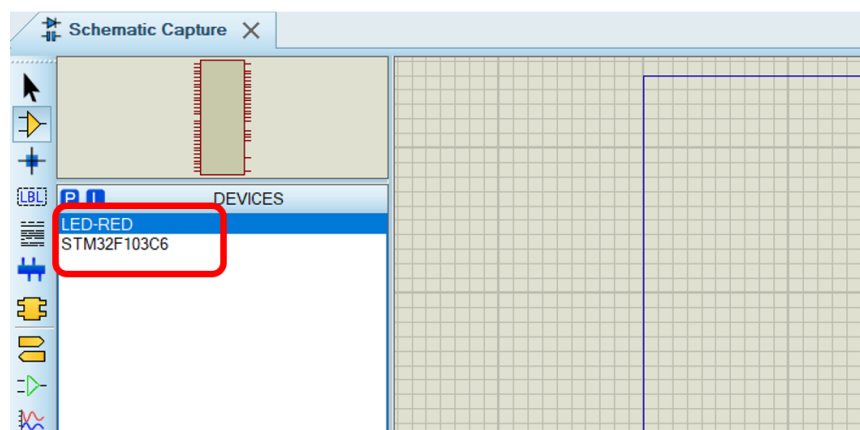


Figure 1.16: STM32 and LED devices in the project

Step 7: Place the components on the schematic. To add power and ground terminals, right-click the schematic and select Place, then Terminal.

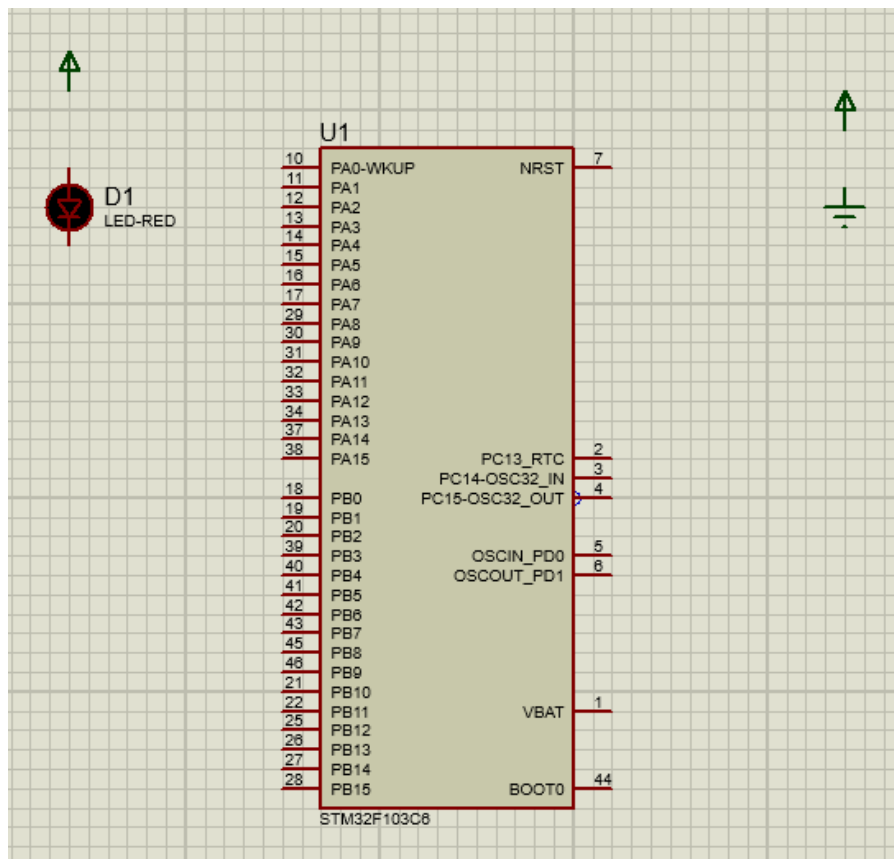


Figure 1.17: Placing components on the schematic

Step 8: Wire the circuit. Connect the negative pin of the LED to PA5 and its positive pin to the power supply. Double-click the power terminal and set its string property to +3.3V.

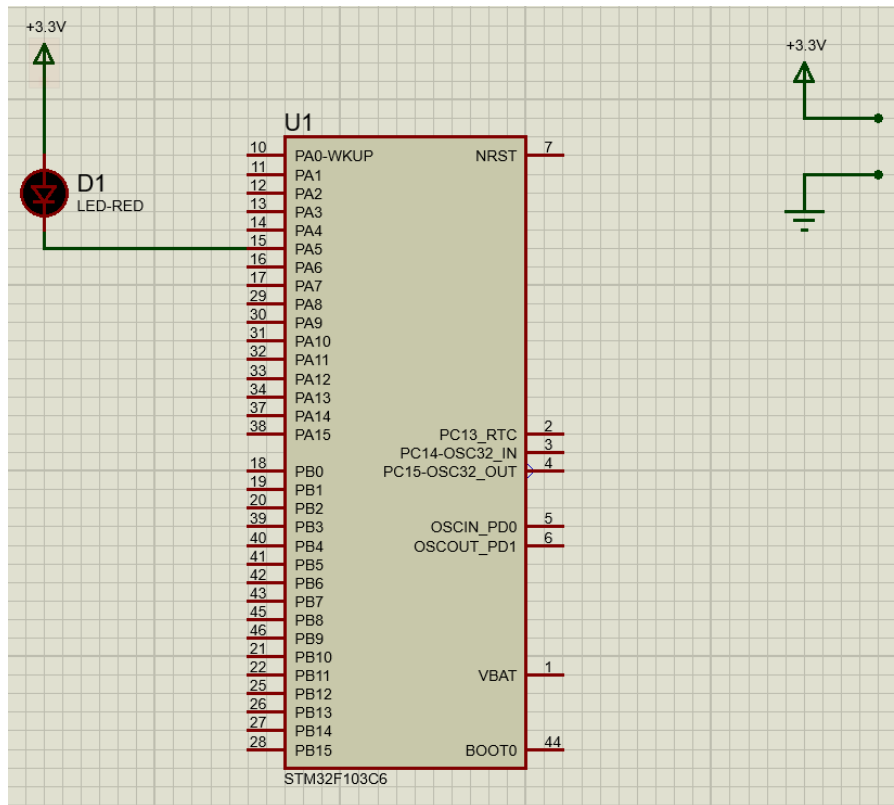


Figure 1.18: Connecting the components and setting the supply to 3.3 V

Add wire labels to the power and ground nets. The STM32 reference pins must also be connected: VDDA to 3.3 V and VSSA to ground.

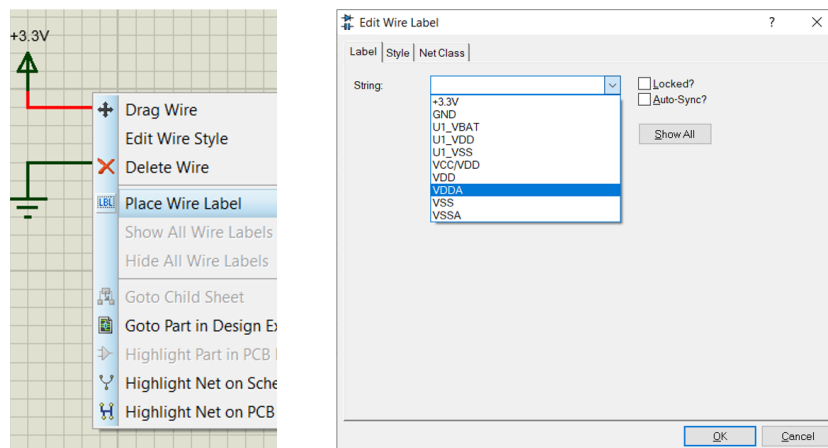


Figure 1.19: Placing labels for the power and ground wires

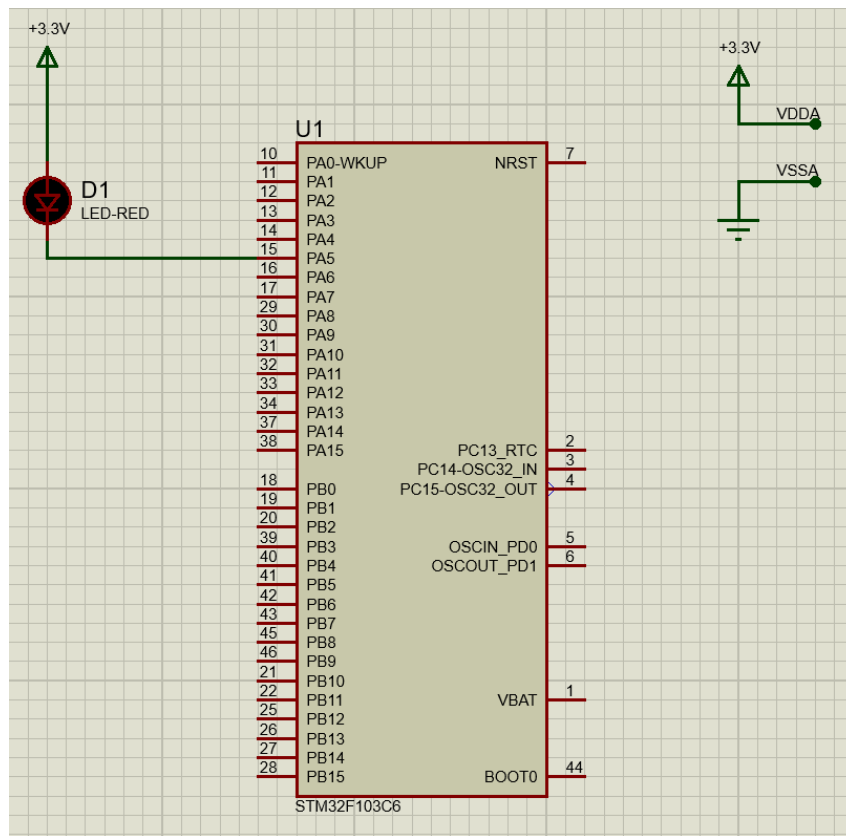


Figure 1.20: Finalized basic schematic

Step 9: Double-click the STM32 and set Program File to the HEX file generated by STM32CubeIDE.

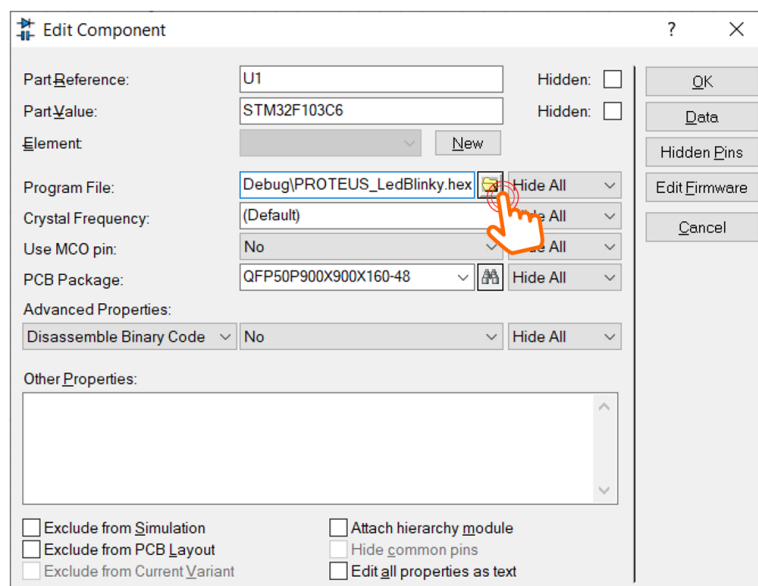


Figure 1.21: Assigning the HEX file to the STM32

Start the simulation from Debug and Run Simulation, or use the quick-access controls at the bottom-left corner. Stop the simulation before modifying either the Proteus schematic or the STM32CubeIDE project.

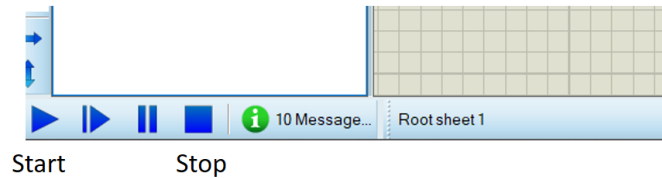


Figure 1.22: Quick-access buttons for starting and stopping simulation

Besides toggling a GPIO pin, the LED can be controlled explicitly using `HAL_GPIO_WritePin`:

```

1 while (1)
2 {
3     HAL_GPIO_WritePin(LED_RED_GPIO_Port, LED_RED_Pin,
4                       GPIO_PIN_SET);
5     HAL_Delay(1000);
6
7     HAL_GPIO_WritePin(LED_RED_GPIO_Port, LED_RED_Pin,
8                       GPIO_PIN_RESET);
9     HAL_Delay(1000);
10 }

```

Program 1.2: Blinking an LED by setting and resetting its GPIO pin

4 Exercise and Report

4.1 Exercise 1

From the Proteus simulation, connect one more LED to pin PA6 of the STM32. The negative pin of the LED is connected to PA6. The suggested component is LED-YELLOW, which can be found in the Proteus device list.

The states of the two LEDs must be switched every two seconds, as shown in Figure 1.23.

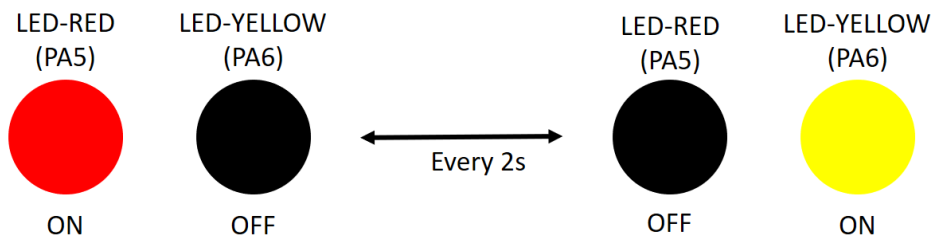


Figure 1.23: State transitions for 2 LEDs

Report 1: Depict the schematic from the Proteus simulation in this report. The figure caption must provide a downloadable link to the Proteus project file, for example, a GitHub link.

Report 2: Present the source code inside the infinite `while` loop. If user-defined functions are used, also present them here and briefly describe them using comments.

Proteus Schematic

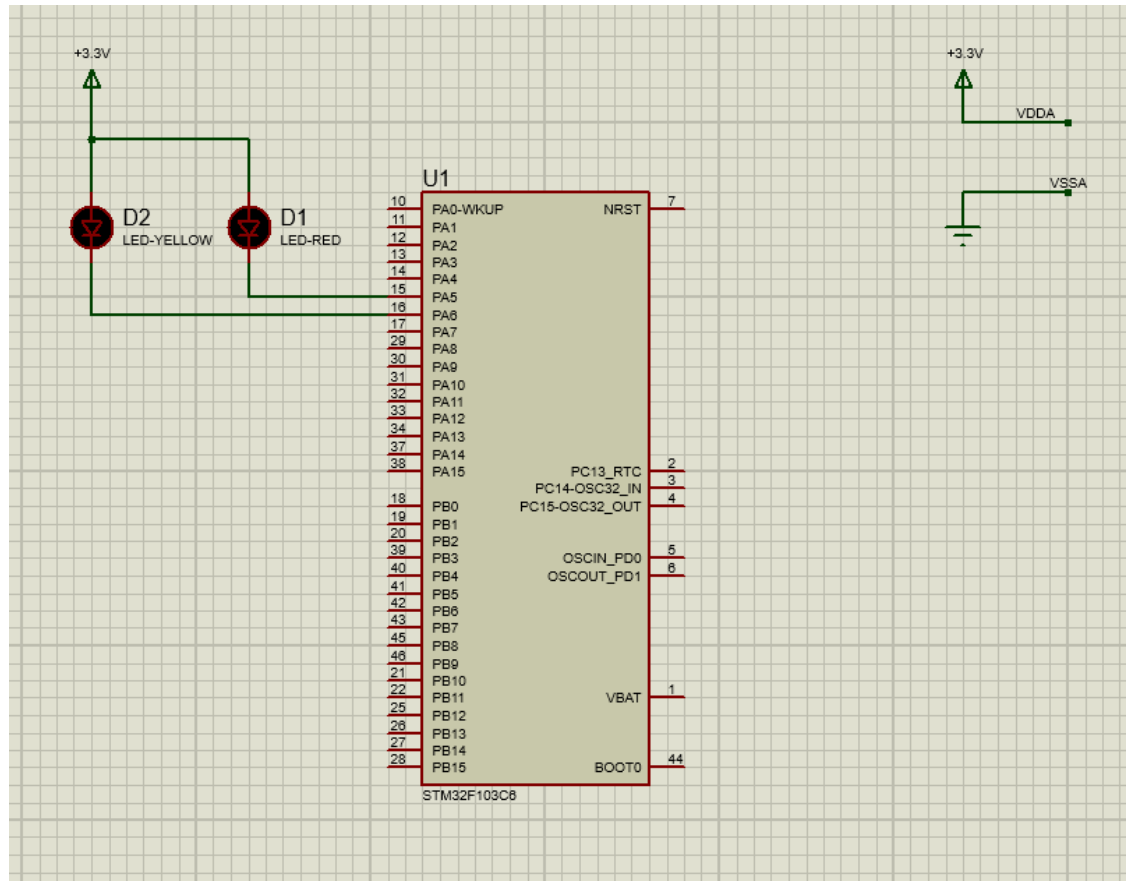


Figure 1.24: Schematic from Proteus for Exercise 1

STM32CubeIDE Configuration

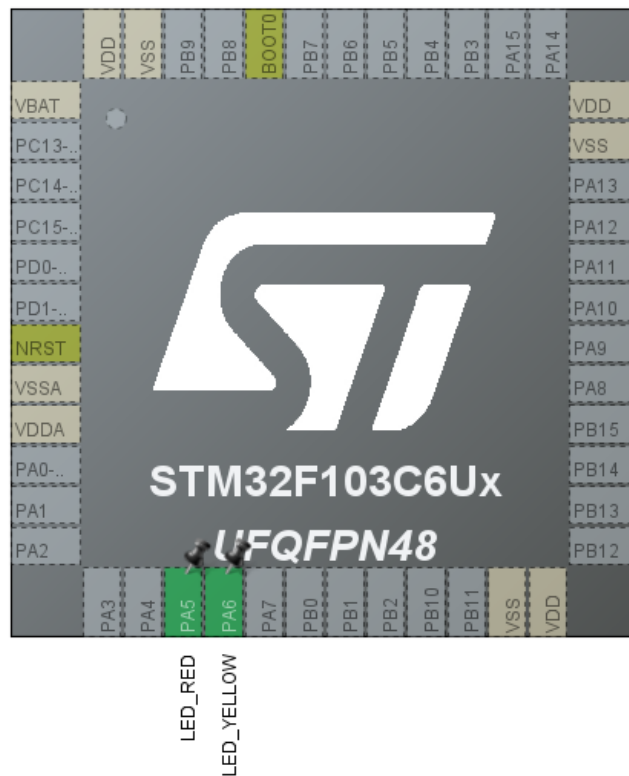


Figure 1.25: GPIO configuration for Exercise 1

STM32CubeIDE Code

```
1 while (1)
2 {
3     HAL_GPIO_WritePin(LED_RED_GPIO_Port, LED_RED_Pin,
4     GPIO_PIN_RESET);
5     HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port, LED_YELLOW_Pin,
6     GPIO_PIN_SET);
7     HAL_Delay(2000);
8     HAL_GPIO_WritePin(LED_RED_GPIO_Port, LED_RED_Pin,
9     GPIO_PIN_SET);
10    HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port, LED_YELLOW_Pin,
11    GPIO_PIN_RESET);
12    HAL_Delay(2000);
13 }
```

Program 1.3: Source code for Exercise 1

4.2 Exercise 2

Extend Exercise 1 to simulate the behavior of a traffic light. Add a third LED, LED-GREEN, connected to PA7. One traffic-light cycle consists of five seconds for RED, two seconds for YELLOW, and three seconds for GREEN. The green LED is also controlled through its negative pin.

Report 1: Present the schematic.

Report 2: Present the source code inside the infinite `while` loop.

Proteus Schematic

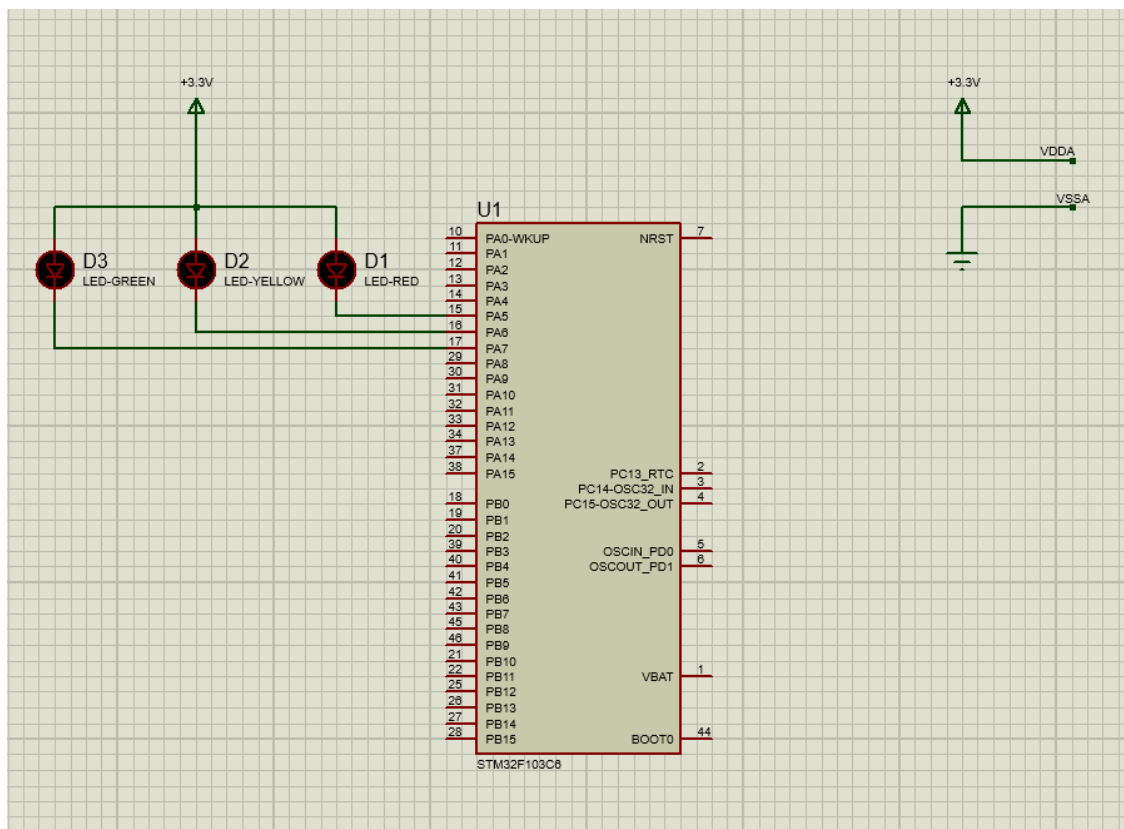


Figure 1.26: Schematic from Proteus for Exercise 2

STM32CubeIDE Configuration

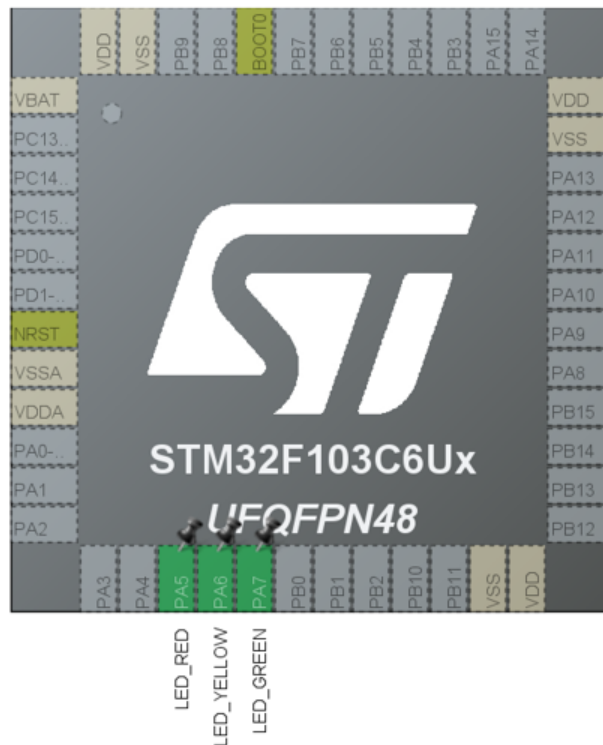


Figure 1.27: GPIO configuration for Exercise 2

STM32CubeIDE Code

```
1 while (1)
2 {
3     HAL_GPIO_WritePin(LED_RED_GPIO_Port, LED_RED_Pin,
4         GPIO_PIN_RESET); // PA5 is labeled as LED_RED
5     HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port, LED_YELLOW_Pin,
6         GPIO_PIN_SET); // PA6 is labeled as LED_YELLOW
7     HAL_GPIO_WritePin(LED_GREEN_GPIO_Port, LED_GREEN_Pin,
8         GPIO_PIN_SET); // PA7 is labeled as LED_GREEN
9     HAL_Delay(5000);
10
11     HAL_GPIO_WritePin(LED_RED_GPIO_Port, LED_RED_Pin,
12         GPIO_PIN_SET);
13     HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port, LED_YELLOW_Pin,
14         GPIO_PIN_RESET);
15     HAL_GPIO_WritePin(LED_GREEN_GPIO_Port, LED_GREEN_Pin,
16         GPIO_PIN_SET);
17     HAL_Delay(2000);
18
19     HAL_GPIO_WritePin(LED_RED_GPIO_Port, LED_RED_Pin,
20         GPIO_PIN_SET);
```

```

14 HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port , LED_YELLOW_Pin ,
    GPIO_PIN_SET);
15 HAL_GPIO_WritePin(LED_GREEN_GPIO_Port , LED_GREEN_Pin ,
    GPIO_PIN_RESET);
16 HAL_Delay(3000);
17 }

```

Program 1.4: Source code for Exercise 2

4.3 Exercise 3

Extend the system to a four-way traffic light. Arrange 12 LEDs in a clear layout to simulate the behavior of traffic lights at an intersection. A reference design is shown in Figure 1.28.

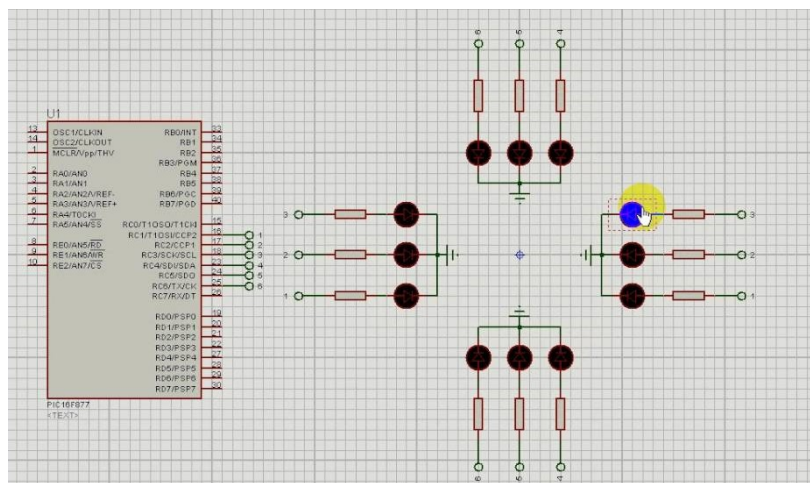


Figure 1.28: Reference design for a 4 way traffic light

Proteus Schematic

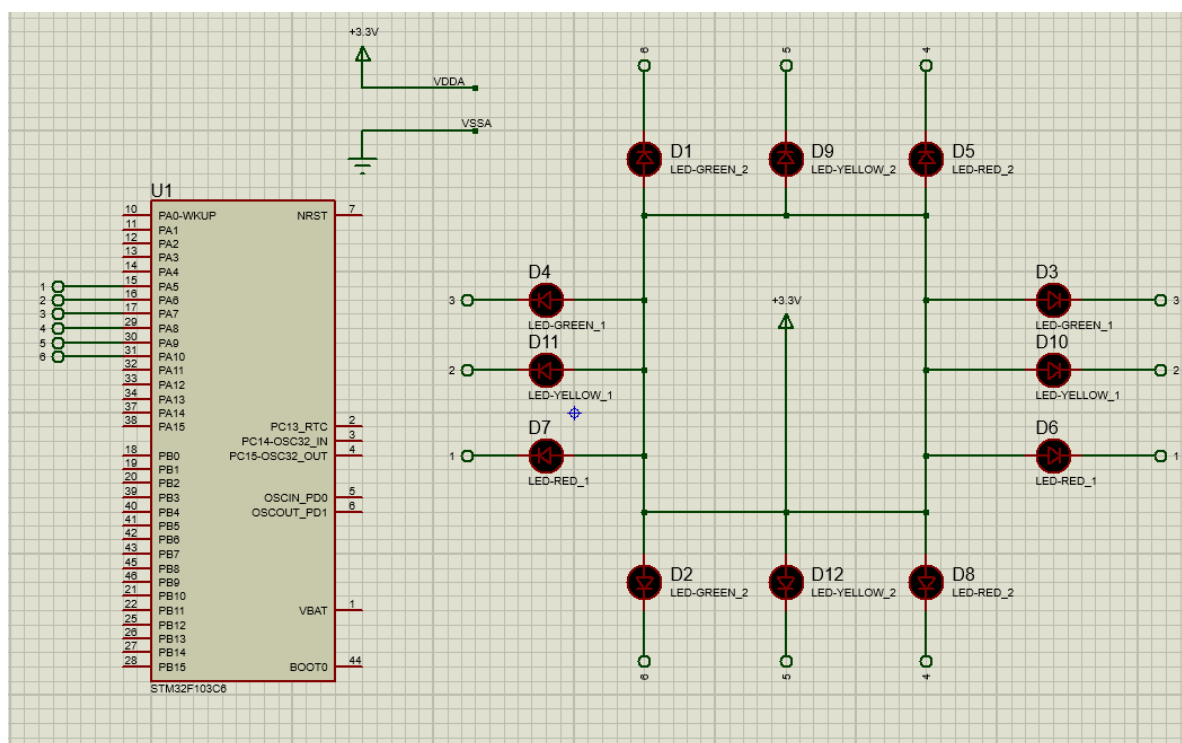


Figure 1.29: Schematic from Proteus for Exercise 3

STM32CubeIDE Configuration

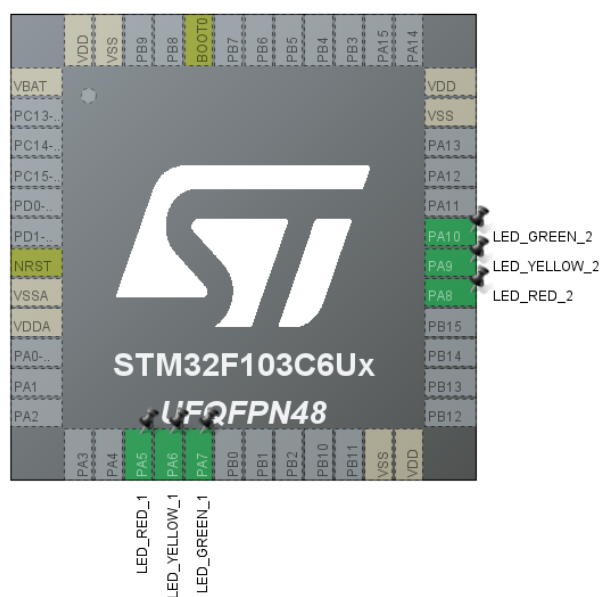


Figure 1.30: GPIO configuration for Exercise 3

STM32CubeIDE Code

```
1 while (1)
2 {
3     // 1: 1: Red(3s) - 2: Green (3s)
4     HAL_GPIO_WritePin(LED_RED_1_GPIO_Port, LED_RED_1_Pin,
5         GPIO_PIN_RESET); // PA5 is labeled as LED_RED_1
6     HAL_GPIO_WritePin(LED_YELLOW_1_GPIO_Port, LED_YELLOW_1_Pin,
7         GPIO_PIN_SET); // PA6 is labeled as LED_YELLOW_1
8     HAL_GPIO_WritePin(LED_GREEN_1_GPIO_Port, LED_GREEN_1_Pin,
9         GPIO_PIN_SET); // PA7 is labeled as LED_GREEN_1
10
11     HAL_GPIO_WritePin(LED_RED_2_GPIO_Port, LED_RED_2_Pin,
12         GPIO_PIN_SET); // PA8 is labeled as LED_RED_2
13     HAL_GPIO_WritePin(LED_YELLOW_2_GPIO_Port, LED_YELLOW_2_Pin,
14         GPIO_PIN_SET); // PA9 is labeled as LED_YELLOW_2
15     HAL_GPIO_WritePin(LED_GREEN_2_GPIO_Port, LED_GREEN_2_Pin,
16         GPIO_PIN_RESET); // PA10 is labeled as LED_GREEN_2
17     HAL_Delay(3000);
18
19     // 2: 1: Red(hold more 2s)      2: Yellow (2s)
20     HAL_GPIO_WritePin(LED_RED_2_GPIO_Port, LED_RED_2_Pin,
21         GPIO_PIN_SET);
22     HAL_GPIO_WritePin(LED_YELLOW_2_GPIO_Port, LED_YELLOW_2_Pin,
23         GPIO_PIN_RESET);
24     HAL_GPIO_WritePin(LED_GREEN_2_GPIO_Port, LED_GREEN_2_Pin,
25         GPIO_PIN_SET);
26     HAL_Delay(2000);
27
28     // 3: 1: Green(3s)      2: Red (3s)
29     HAL_GPIO_WritePin(LED_RED_1_GPIO_Port, LED_RED_1_Pin,
30         GPIO_PIN_SET);
31     HAL_GPIO_WritePin(LED_YELLOW_1_GPIO_Port, LED_YELLOW_1_Pin,
32         GPIO_PIN_SET);
33     HAL_GPIO_WritePin(LED_GREEN_1_GPIO_Port, LED_GREEN_1_Pin,
34         GPIO_PIN_RESET);
35
36     HAL_GPIO_WritePin(LED_RED_2_GPIO_Port, LED_RED_2_Pin,
37         GPIO_PIN_RESET);
38     HAL_GPIO_WritePin(LED_YELLOW_2_GPIO_Port, LED_YELLOW_2_Pin,
39         GPIO_PIN_SET);
40     HAL_GPIO_WritePin(LED_GREEN_2_GPIO_Port, LED_GREEN_2_Pin,
41         GPIO_PIN_SET);
42     HAL_Delay(3000);
43
44     // 4: 1: Yellow(2s)      2: Red (hold more 2s)
45     HAL_GPIO_WritePin(LED_RED_1_GPIO_Port, LED_RED_1_Pin,
46         GPIO_PIN_SET);
```

```

31 HAL_GPIO_WritePin(LED_YELLOW_1_GPIO_Port , LED_YELLOW_1_Pin ,
    GPIO_PIN_RESET);
32 HAL_GPIO_WritePin(LED_GREEN_1_GPIO_Port , LED_GREEN_1_Pin ,
    GPIO_PIN_SET);
33 HAL_Delay(2000);
34 }

```

Program 1.5: Source code for Exercise 3

4.4 Exercise 4

Add one seven-segment LED to the schematic from Exercise 3. In Proteus, the component can be found using the keyword 7SEG-COM-ANODE. Connect its common pin to the power supply and connect segments A through G to PB0 through PB6. Because this is a common-anode display, an STM32 output at logic 0 (0 V) turns on the corresponding segment.

Implement the function `display7SEG(int num)`. Its input ranges from 0 to 9, and the expected segment patterns are shown in Figure 1.31.

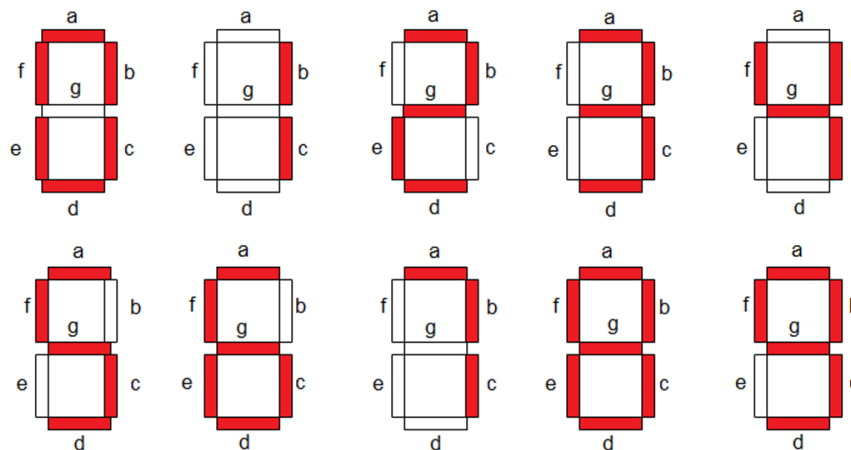


Figure 1.31: Display a number on 7 segment LED

Test the function in the infinite loop using a counter that repeatedly counts from 0 to 9, with a delay of one second between consecutive values.

Report 1: Present the schematic.

Report 2: Present the source code of the `display7SEG` function.

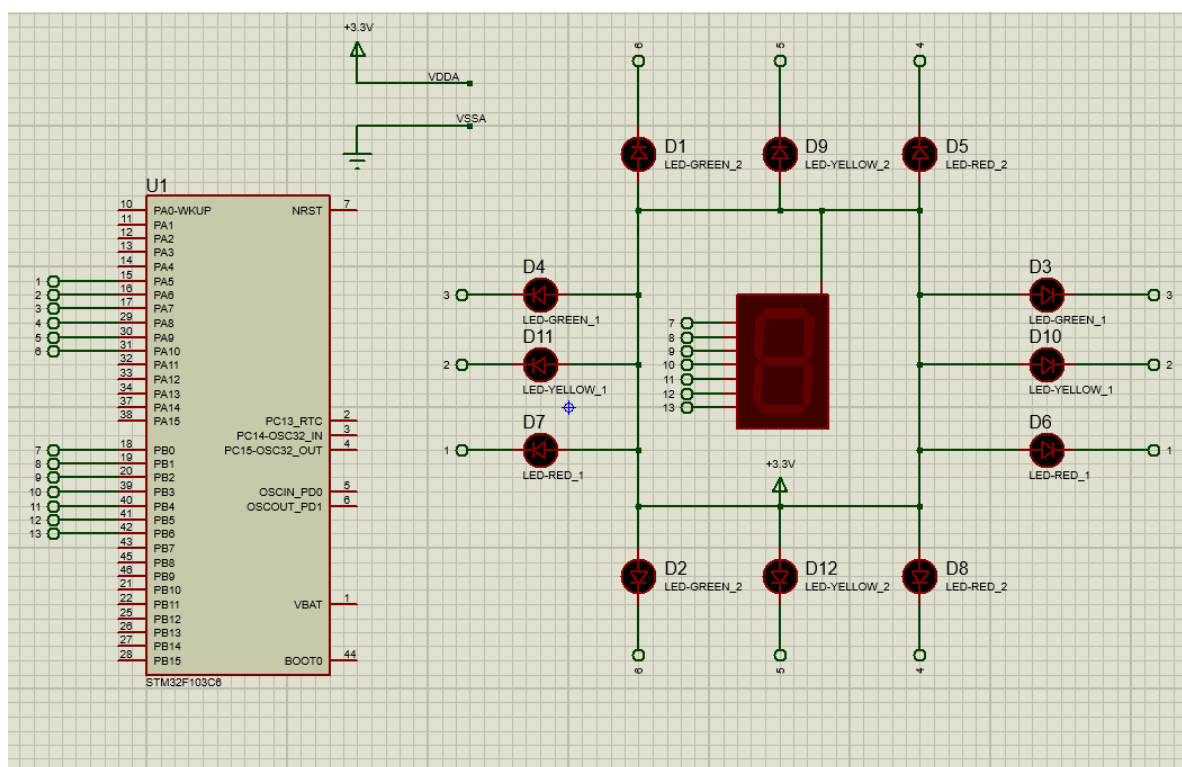


Figure 1.32: Schematic from Proteus for Exercise 4

STM32CubeIDE Configuration

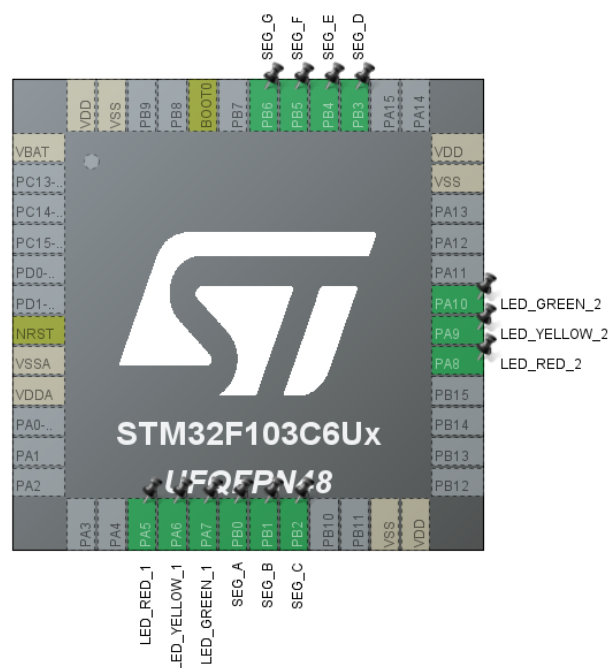


Figure 1.33: GPIO configuration for Exercise 4

STM32CubeIDE Code

```
1 void display7SEG(int num) {
2     uint16_t seg_pins[7] = {
3         SEG_A_Pin, SEG_B_Pin, SEG_C_Pin,
4         SEG_D_Pin, SEG_E_Pin, SEG_F_Pin, SEG_G_Pin
5     }; // PB0-PB6 are labeled as SEG_A-SEG_G respectively
6
7     uint8_t seg_data[10][7] = {
8         {0,0,0,0,0,0,1}, // 0
9         {1,0,0,1,1,1,1}, // 1
10        {0,0,1,0,0,1,0}, // 2
11        {0,0,0,0,1,1,0}, // 3
12        {1,0,0,1,1,0,0}, // 4
13        {0,1,0,0,1,0,0}, // 5
14        {0,1,0,0,0,0,0}, // 6
15        {0,0,0,1,1,1,1}, // 7
16        {0,0,0,0,0,0,0}, // 8
17        {0,0,0,0,1,0,0} // 9
18    };
19
20    if (num >= 0 && num <= 9) {
21        for (int i = 0; i < 7; i++) {
22            if (seg_data[num][i] == 0) {
23                HAL_GPIO_WritePin(GPIOB, seg_pins[i],
24                GPIO_PIN_RESET);
25            }
26            else {
27                HAL_GPIO_WritePin(GPIOB, seg_pins[i],
28                GPIO_PIN_SET);
29            }
30        }
31    }
32
33    int counter = 0;
34    while (1){
35        if (counter >= 10) counter = 0;
36        display7SEG(counter++);
37        HAL_Delay(1000);
38    }
```

Program 1.6: Source code for Exercise 4

4.5 Exercise 5

Integrate the seven-segment LED into the four-way traffic-light system. Use the seven-segment LED to display the countdown value. Only the source code is required in this exercise; the display7SEG function from Exercise 4 may be reused.

Proteus Schematic

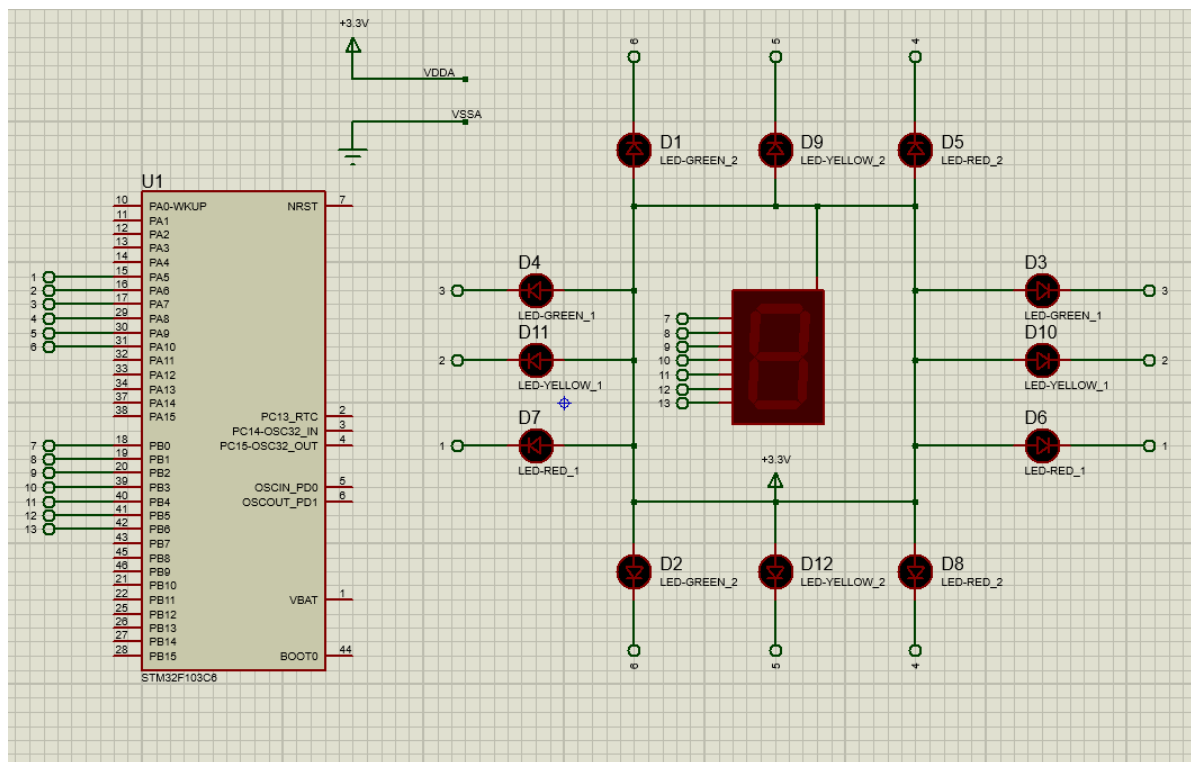


Figure 1.34: Schematic from Proteus for Exercise 5

STM32CubeIDE Configuration

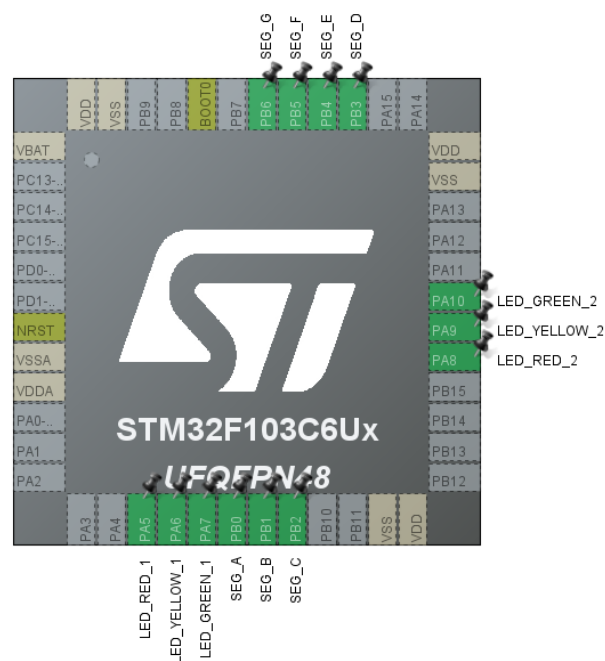


Figure 1.35: GPIO configuration for Exercise 5

STM32CubeIDE Code

```
1 void display7SEG(int num) {
2     uint16_t seg_pins[7] = {
3         SEG_A_Pin, SEG_B_Pin, SEG_C_Pin,
4         SEG_D_Pin, SEG_E_Pin, SEG_F_Pin, SEG_G_Pin
5     }; // PB0-PB6 are labeled as SEG_A-SEG_G respectively
6
7     uint8_t seg_data[10][7] = {
8         {0,0,0,0,0,0,1}, // 0
9         {1,0,0,1,1,1,1}, // 1
10        {0,0,1,0,0,1,0}, // 2
11        {0,0,0,0,1,1,0}, // 3
12        {1,0,0,1,1,0,0}, // 4
13        {0,1,0,0,1,0,0}, // 5
14        {0,1,0,0,0,0,0}, // 6
15        {0,0,0,1,1,1,1}, // 7
16        {0,0,0,0,0,0,0}, // 8
17        {0,0,0,0,1,0,0} // 9
18    };
19
20    if (num >= 0 && num <= 9) {
21        for (int i = 0; i < 7; i++) {
22            if (seg_data[num][i] == 0) {
23                HAL_GPIO_WritePin(GPIOB, seg_pins[i],
24                GPIO_PIN_RESET);
25            }
26            else {
27                HAL_GPIO_WritePin(GPIOB, seg_pins[i], GPIO_PIN_SET)
28            };
29        }
30    }
31
32    int t = 0;
33    int counter1 = 0;
34    while (1)
35    {
36        if (t < 5) {
37            HAL_GPIO_WritePin(LED_RED_1_GPIO_Port, LED_RED_1_Pin,
38            GPIO_PIN_RESET); // PA5 is labeled as LED_RED_1
39            HAL_GPIO_WritePin(LED_YELLOW_1_GPIO_Port, LED_YELLOW_1_Pin,
40            GPIO_PIN_SET); // PA6 is labeled as LED_YELLOW_1
41            HAL_GPIO_WritePin(LED_GREEN_1_GPIO_Port, LED_GREEN_1_Pin,
42            GPIO_PIN_SET); // PA7 is labeled as LED_GREEN_1
43            counter1 = 5 - t;
44
45            if (t < 3) {
```

```

43     HAL_GPIO_WritePin(LED_RED_2_GPIO_Port, LED_RED_2_Pin,
GPIO_PIN_SET); // PA8 is labeled as LED_RED_2
44     HAL_GPIO_WritePin(LED_YELLOW_2_GPIO_Port,
LED_YELLOW_2_Pin, GPIO_PIN_SET); // PA9 is labeled as
LED_YELLOW_2
45     HAL_GPIO_WritePin(LED_GREEN_2_GPIO_Port, LED_GREEN_2_Pin,
GPIO_PIN_RESET); // PA10 is labeled as LED_GREEN_2
46 }
47 else {
48     HAL_GPIO_WritePin(LED_RED_2_GPIO_Port, LED_RED_2_Pin,
GPIO_PIN_SET);
49     HAL_GPIO_WritePin(LED_YELLOW_2_GPIO_Port,
LED_YELLOW_2_Pin, GPIO_PIN_RESET);
50     HAL_GPIO_WritePin(LED_GREEN_2_GPIO_Port, LED_GREEN_2_Pin,
GPIO_PIN_SET);
51 }
52 }
53 else if (t < 8) {
54     HAL_GPIO_WritePin(LED_RED_1_GPIO_Port, LED_RED_1_Pin,
GPIO_PIN_SET);
55     HAL_GPIO_WritePin(LED_YELLOW_1_GPIO_Port, LED_YELLOW_1_Pin,
GPIO_PIN_SET);
56     HAL_GPIO_WritePin(LED_GREEN_1_GPIO_Port, LED_GREEN_1_Pin,
GPIO_PIN_RESET);
57
58     HAL_GPIO_WritePin(LED_RED_2_GPIO_Port, LED_RED_2_Pin,
GPIO_PIN_RESET);
59     HAL_GPIO_WritePin(LED_YELLOW_2_GPIO_Port, LED_YELLOW_2_Pin,
GPIO_PIN_SET);
60     HAL_GPIO_WritePin(LED_GREEN_2_GPIO_Port, LED_GREEN_2_Pin,
GPIO_PIN_SET);
61     counter1 = 8 - t;
62 }
63 else {
64     HAL_GPIO_WritePin(LED_RED_1_GPIO_Port, LED_RED_1_Pin,
GPIO_PIN_SET);
65     HAL_GPIO_WritePin(LED_YELLOW_1_GPIO_Port, LED_YELLOW_1_Pin,
GPIO_PIN_RESET);
66     HAL_GPIO_WritePin(LED_GREEN_1_GPIO_Port, LED_GREEN_1_Pin,
GPIO_PIN_SET);
67
68     HAL_GPIO_WritePin(LED_RED_2_GPIO_Port, LED_RED_2_Pin,
GPIO_PIN_RESET);
69     HAL_GPIO_WritePin(LED_YELLOW_2_GPIO_Port, LED_YELLOW_2_Pin,
GPIO_PIN_SET);
70     HAL_GPIO_WritePin(LED_GREEN_2_GPIO_Port, LED_GREEN_2_Pin,
GPIO_PIN_SET);
71     counter1 = 10 - t;
72 }

```

```

73
74     display7SEG(counter1);
75     HAL_Delay(1000);
76
77     t++;
78     if (t >= 10) {
79         t = 0;
80     }
81 }

```

Program 1.7: Source code for Exercise 5

4.6 Exercise 6

Design a new Proteus schematic to simulate an analog clock with 12 positions. Connect the 12 LEDs to pins PA4 through PA15 of the STM32 and arrange them as shown in Figure 1.36.

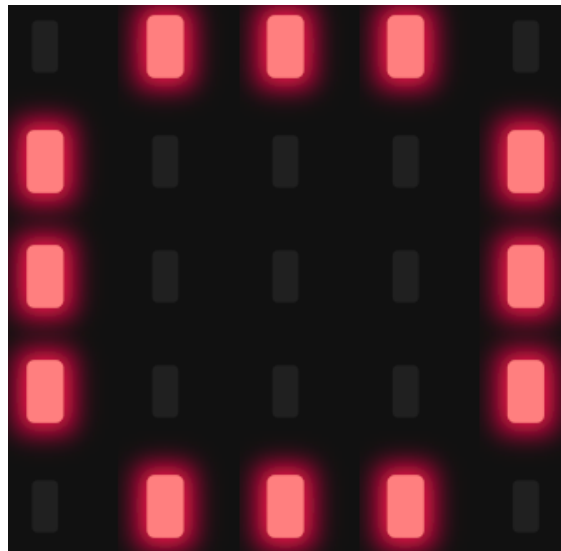


Figure 1.36: 12 LEDs for an analog clock

Report 1: Present the schematic.

Report 2: Implement a simple program that tests every LED connection by turning on each LED sequentially.

Proteus Schematic

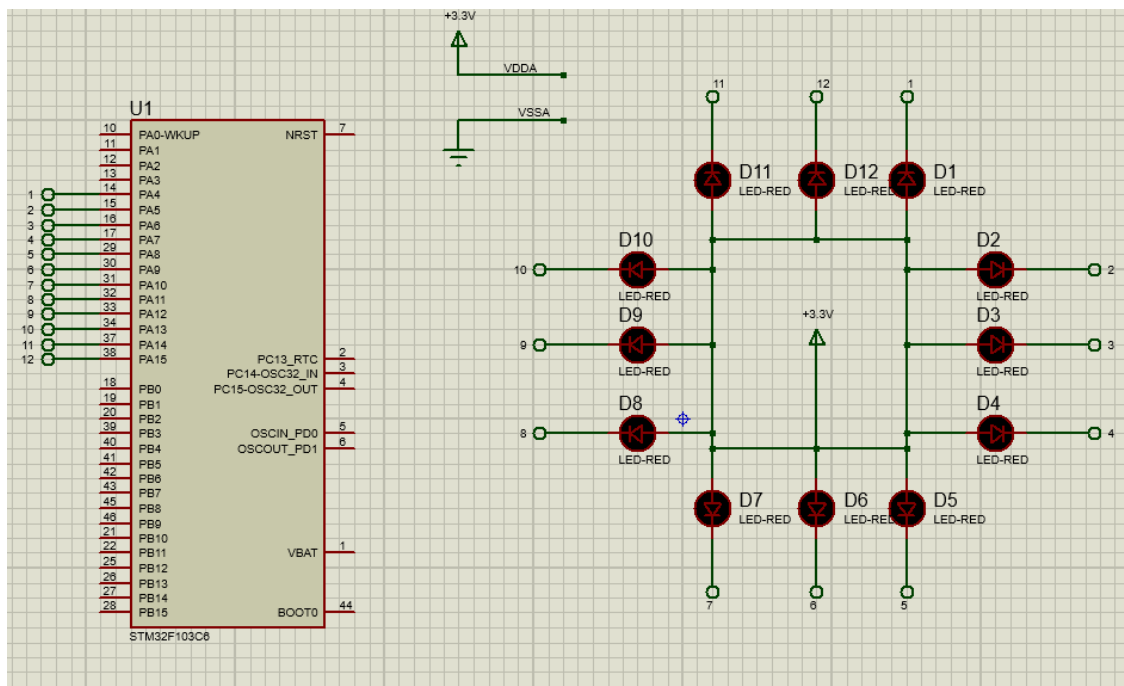


Figure 1.37: Schematic from Proteus for Exercise 6

STM32CubeIDE Configuration

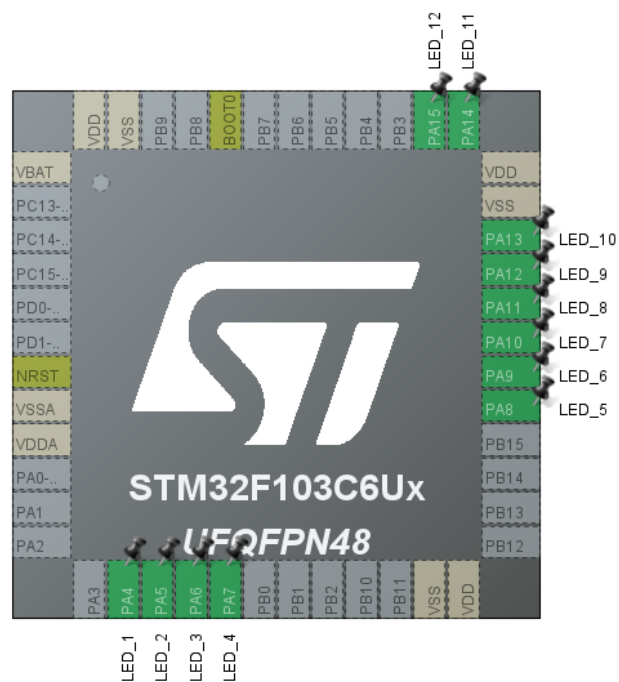


Figure 1.38: GPIO configuration for Exercise 6

```

1 uint16_t clock_pins[12] = {
2     LED_1_Pin, LED_2_Pin, LED_3_Pin, LED_4_Pin,
3     LED_5_Pin, LED_6_Pin, LED_7_Pin, LED_8_Pin,
4     LED_9_Pin, LED_10_Pin, LED_11_Pin, LED_12_Pin
5 }; // PA4-PA15 are labeled as LED_1-LED_12 respectively
6 while (1)
7 {
8     for (int i = 0; i < 12; i++) {
9         for (int j = 0; j < 12; j++) {
10             HAL_GPIO_WritePin(GPIOA, clock_pins[j],
11             GPIO_PIN_SET);
12             HAL_GPIO_WritePin(GPIOA, clock_pins[i], GPIO_PIN_RESET)
13         };
14         HAL_Delay(1000);
15     }
16 }

```

Program 1.8: Source code for Exercise 6

4.7 Exercise 7

Implement a function named `clearAllClock()` that turns off all 12 LEDs. Present the source code of this function.

STM32CubeIDE Code

```

1 void clearAllClock()
2 {
3     //TODO
4 }

```

Program 1.9: Function Implementation

STM32CubeIDE Code

```

1 uint16_t clock_pins[12] = {
2     LED_1_Pin, LED_2_Pin, LED_3_Pin, LED_4_Pin,
3     LED_5_Pin, LED_6_Pin, LED_7_Pin, LED_8_Pin,
4     LED_9_Pin, LED_10_Pin, LED_11_Pin, LED_12_Pin
5 };
6 void clearAllClock(void) {
7     for (int i = 0; i < 12; i++) {
8         HAL_GPIO_WritePin(GPIOA, clock_pins[i], GPIO_PIN_SET);
9     }
10 }
11 while (1)

```

```

12 {
13     for (int i = 0; i < 12; i++) {
14         HAL_GPIO_WritePin(GPIOA, clock_pins[i], GPIO_PIN_RESET);
15     }
16     HAL_Delay(1000);
17     clearAllClock();
18     HAL_Delay(1000);
19 }

```

Program 1.10: Source code for Exercise 7

4.8 Exercise 8

Implement a function named `setNumberOnClock(int num)`. The input ranges from 0 to 11, and the corresponding LED must be turned on. Present the source code of this function.

STM32CubeIDE Code

```

1 uint16_t clock_pins[12] = {
2     LED_1_Pin, LED_2_Pin, LED_3_Pin, LED_4_Pin,
3     LED_5_Pin, LED_6_Pin, LED_7_Pin, LED_8_Pin,
4     LED_9_Pin, LED_10_Pin, LED_11_Pin, LED_12_Pin
5 }; // PA4-PA15 are labeled as LED_1-LED_12 respectively
6
7 void clearAllClock(void) {
8     for (int i = 0; i < 12; i++) {
9         HAL_GPIO_WritePin(GPIOA, clock_pins[i], GPIO_PIN_SET);
10    }
11 }
12 void setNumberOnClock(int num) {
13     if (num >= 0 && num < 12) {
14         int index = (num == 0) ? 11 : (num - 1);
15         HAL_GPIO_WritePin(GPIOA, clock_pins[index],
16             GPIO_PIN_RESET);
17     }
18 }
19 while (1)
20 {
21     for (int i = 0; i < 12; i++) {
22         clearAllClock();
23         setNumberOnClock(i);
24         HAL_Delay(1000);
25     }
26 }

```

Program 1.11: Source code for Exercise 8

4.9 Exercise 9

Implement a function named `clearNumberOnClock(int num)`. The input ranges from 0 to 11, and the corresponding LED must be turned off. Present the source code of this function.

STM32CubeIDE Code

```
1 uint16_t clock_pins[12] = {
2     LED_1_Pin, LED_2_Pin, LED_3_Pin, LED_4_Pin,
3     LED_5_Pin, LED_6_Pin, LED_7_Pin, LED_8_Pin,
4     LED_9_Pin, LED_10_Pin, LED_11_Pin, LED_12_Pin
5 };
6 void setNumberOnClock(int num) {
7     if (num >= 0 && num < 12) {
8         int index = (num == 0) ? 11 : (num - 1);
9         HAL_GPIO_WritePin(GPIOA, clock_pins[index],
10        GPIO_PIN_RESET);
11    }
12 }
13 void clearNumberOnClock(int num) {
14     if (num >= 0 && num < 12) {
15         int index = (num == 0) ? 11 : (num - 1);
16         HAL_GPIO_WritePin(GPIOA, clock_pins[index],
17        GPIO_PIN_SET);
18    }
19 }
20 while (1)
21 {
22     for (int i = 0; i < 12; i++) {
23         setNumberOnClock(i);
24     }
25     HAL_Delay(1000);
26     for (int i = 0; i < 12; i++) {
27         clearNumberOnClock(i);
28         HAL_Delay(1000);
29     }
30 }
```

Program 1.12: Source code for Exercise 9

4.10 Exercise 10

Integrate the complete system and use the 12 LEDs to display an analog clock. At any given time, only three LEDs should be on to represent the hour, minute, and second information.

Proteus Schematic

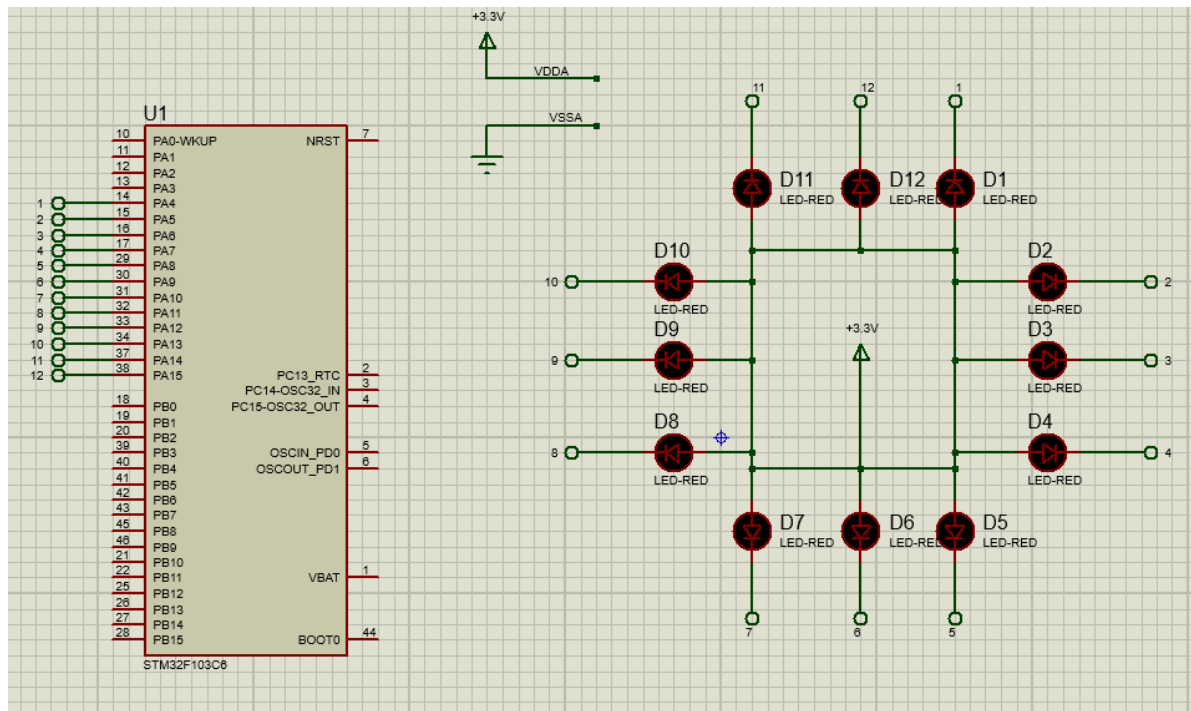


Figure 1.39: Schematic from Proteus for Exercise 10

STM32CubeIDE Code

```
1 uint16_t clock_pins[12] = {
2     LED_1_Pin, LED_2_Pin, LED_3_Pin, LED_4_Pin,
3     LED_5_Pin, LED_6_Pin, LED_7_Pin, LED_8_Pin,
4     LED_9_Pin, LED_10_Pin, LED_11_Pin, LED_12_Pin
5 };
6 void clearAllClock(void) {
7
8     for (int i = 0; i < 12; i++) {
9         HAL_GPIO_WritePin(GPIOA, clock_pins[i], GPIO_PIN_SET);
10    }
11 }
12 void updateClock(int hour, int minute, int second) {
13     int h = hour % 12;
14     int m = minute / 5;
15     int s = second / 5;
16
17     for (int i = 0; i < 12; i++) {
18         int index = (i == 0) ? 11 : (i - 1);
19
20         if (i == h || i == m || i == s) {
21             HAL_GPIO_WritePin(GPIOA, clock_pins[index],
22                               GPIO_PIN_RESET);
```

```

22     }
23     else {
24         HAL_GPIO_WritePin(GPIOA, clock_pins[index],
25         GPIO_PIN_SET);
26     }
27 }
28 int hour = 15;
29 int minute = 47;
30 int second = 0;
31 clearAllClock();
32 while (1)
33 {
34     updateClock(hour, minute, second);
35     HAL_Delay(1000);
36     second++;
37     if (second >= 60) {
38         second = 0;
39         minute++;
40         if (minute >= 60) {
41             minute = 0;
42             hour++;
43             if (hour >= 24) {
44                 hour = 0;
45             }
46         }
47     }
48 }

```

Program 1.13: Source code for Exercise 10